1. First Terraform ante enti?

Suppose nuvvu oka company lo DevOps engineer.
Company ki video application undi.
Users application open chesthe:
User
↓
Video App
↓
Videos watch chestaru
Aa application AWS servers meeda run avvali.
AWS lo manaki chala infrastructure kavali:
Internet
↓
Network
↓
Servers
↓
Database
↓
Storage
Ivi AWS console lo manually create cheyyachu.
Example:
AWS Console
↓
Create VPC
↓
Create Subnet
↓
Create Security Group
↓
Create EC2
↓
Create Database
↓
Create Load Balancer
Kani company lo 100 servers, 10 applications, 3 environments unte manually create cheyyadam difficult.
Anduke Terraform.
Terraform ante:
Infrastructure ni code raasi create/manage cheyyadaniki use chese tool.
Ante AWS console lo buttons click cheyyakunda, code raasi AWS ki cheptham:
"Naaku ilaanti infrastructure kavali."
Terraform aa code ni AWS ki pampinchi infrastructure create/manage chestundi.

---

2. Terraform lo "code" ante enti?
Nuvvu programming lo:
if
for
function
lantivi vinnavemo.
Terraform lo main ga manam ila rastam:

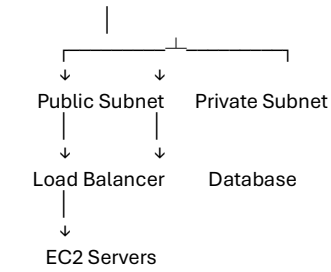
```
resource "aws_instance" "app" {
  instance_type = "t3.micro"
}
```

Idi Terraform language lo oka resource block.
Meaning:
AWS lo oka EC2 instance ni create/manage cheyyi.
So Terraform code ante basically:
AWS/Azure/GCP etc. lo infrastructure ela undalo describe chese configuration.

---

3. First AWS infrastructure enti ani teliyali
Terraform ardham avvalante AWS basics konchem ardham avvali.
Mana video application example:





Ippudu prati word enti chuddam.

---

4. VPC ante enti?
VPC = Virtual Private Cloud
Simple ga:
AWS lo mana application kosam create chesukunna private network.
Real life example:
Oka company office imagine cheyyi.

```
Company Office
│
├── Reception
├── Employees area
├── Server room
└── Private rooms
```

Office ki own boundaries untayi.
AWS lo application kosam alanti boundary:
VPC
VPC lopala mana resources untayi.

```
VPC
│
├── Servers
├── Database
├── Load Balancer
└── Other resources
```

Terraform tho VPC create cheyyachu.
Example:

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
```

Meaning:
AWS lo 10.0.0.0/16 network range tho oka VPC create cheyyi.

---

5. Subnet ante enti?
VPC oka pedda office anuko.
Office ni different areas ga divide chestham.
AWS lo VPC ni smaller networks ga divide chestham.
Avi Subnets.

```
VPC
│
├── Public Subnet
│
└── Private Subnet
```

Public subnet
Internet nunchi access avvalsina resources kosam.
Example:
Load Balancer
Private subnet
Direct internet access avvakudadani resources kosam.
Example:
Database
So:

```
VPC
│
├── Public Subnet
│     ↓
│   Load Balancer
│
└── Private Subnet
      ↓
    Database
```

---

6. EC2 ante enti?
EC2 = AWS virtual server.
Simple ga:
AWS lo rent ki teesukunna computer/server.
Mana video application run cheyyali.
So:
EC2
servers create chestham.
Example:
EC2 #1
EC2 #2
EC2 #3
Application code ee servers meeda run avvachu.
Terraform:

```
resource "aws_instance" "app" {
  instance_type = "t3.micro"
}
```

Meaning:
AWS lo t3.micro size virtual server create/manage cheyyi.

---

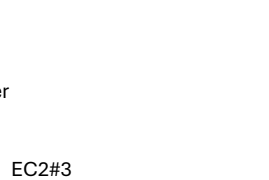
7. Database ante enti?
Application ki information store cheyyali.
Example:
User name
Email
Password information
Subscriptions
Watch history
Ivi database lo store chestham.
AWS lo example:
RDS
RDS ante AWS managed database service.
Terraform tho:

```
resource "aws_db_instance" "database" {
  engine = "mysql"
}
```

lanti code tho database infrastructure create/manage cheyyachu.

---

8. Load Balancer ante enti?
Suppose mana daggara 3 servers unnayi.
EC2 #1
EC2 #2
EC2 #3
1000 users application ki vacharu.
Users ni oka server meedaki matrame pampakunda requests ni distribute cheyyali.
Load Balancer:



Terraform tho Load Balancer kuda create cheyyachu.

---

9. Storage ante enti?
Videos chala large files.
Example:
movie1.mp4
movie2.mp4
trailer.mp4
poster.jpg
Ivi store cheyyadaniki AWS lo S3 lantivi use cheyyachu.
S3

```
│
├── movie1.mp4
├── movie2.mp4
├── trailer.mp4
└── posters
```

Terraform tho S3 bucket create/manage cheyyachu.

10. Ippudu Terraform actual purpose ardham chesko
Mana application ki AWS lo:
VPC
Subnet
EC2
Database
Load Balancer
Storage
kavali.
Veetini AWS Console lo manually create cheyyachu.
Kani Terraform lo manam code rastam:
Terraform Code
↓
Terraform
↓
AWS
↓
VPC
Subnet
EC2
Database
Load Balancer
Storage
Idi Terraform main job.

---

11. Ippudu project folder ki veldam
Nuvvu mundu chusina:
video-app/
│
├── main.tf
├── variables.tf
└── terraform.tfvars
Idi enti?
Idi just Terraform project files.
main.tf
Meaning:
Infrastructure em kavali?
variables.tf
Meaning:
Infrastructure ki change ayye inputs em kavali?
terraform.tfvars
Meaning:
Aa inputs ki actual values enti?

---

12. Oka very small example
Suppose:
Naaku AWS lo 2 EC2 servers kavali.
Manam variables create chestham.
variables.tf

```
variable "instance_count" {
  type = number
}

variable "instance_type" {
  type = string
}
```

Meaning:
instance_count → enni servers?
instance_type → server size enti?

---

terraform.tfvars

```
instance_count = 2
instance_type  = "t3.micro"
```

Meaning:
Enni servers? → 2
Server size? → t3.micro

---

main.tf

```
resource "aws_instance" "app" {
```

```
  count       = var.instance_count
  instance_type = var.instance_type

}
```

Meaning:

```
var.instance_count
      ↓
      2
      ↓
```

2 EC2 servers create cheyyi

And:

```
var.instance_type
      ↓
  t3.micro
      ↓
```

server size = t3.micro

So final:

```
AWS
│
 ├── EC2 #1 → t3.micro
 └── EC2 #2 → t3.micro
```

---

13. Ippudu modules ante enti?

Idi nee main confusion.

Suppose company lo 10 applications unnayi.

Prati application ki VPC kavali.

Manam VPC code ni 10 times rayala?

App 1 → VPC code

App 2 → VPC code

App 3 → VPC code

...

App 10 → VPC code

Waste.

Instead oka sari VPC code rayi:

```
modules/
 └── vpc/
```

Aa code ni different applications reuse chestayi.

Module ante reusable Terraform code.

Simple analogy:

Oka company lo common printer undi. Prati employee separate printer konukkodu. Common printer ni use chestaru.

Same way:

Common Terraform infrastructure code ni module ga create chesi different projects reuse chestayi.

---

14. modules/vpc lopala em untundi?

```
modules/
 └── vpc/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```

vpc/main.tf

VPC create cheyyadaniki code:

```
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
}
```

Meaning:

VPC create cheyyi.

vpc/variables.tf

```
variable "vpc_cidr" {
  type = string
}
```

Meaning:

VPC ki CIDR input kavali.

vpc/outputs.tf

```
output "vpc_id" {
  value = aws_vpc.main.id
}
```

Meaning:

Create aina VPC ID ni module bayata use cheyyadaniki ivvu.

---

15. Mari module ni ela use chestham?

Main project lo:

```
module "vpc" {

 source = "../../modules/vpc"

 vpc_cidr = "10.0.0.0/16"

}
```

Meaning:
modules/vpc lo unna code ni use cheyyi.
So:

```
main project
   ↓
module "vpc"
   ↓
modules/vpc
   ↓
vpc/main.tf
   ↓
AWS VPC
```

---

16. Same thing EC2 ki

```
modules/
│
├── vpc/
│
├── ec2/
│
├── rds/
│
└── alb/
```

Meaning:
vpc → VPC related code
ec2 → EC2 related code
rds → Database related code
alb → Load Balancer related code
Ivi Terraform ki compulsory names kaavu.
Company ki organization choice.

---

17. environments ante enti?
Ippudu next big concept.
Company application ki usually different environments untayi.
DEV
STAGE
PROD
DEV
Developers testing chestaru.
dev
STAGE
Production ki mundu testing.
stage
PROD
Real users use chese environment.
prod
Problem enti?
Dev ki:
2 small servers
kavali.
Prod ki:
10 bigger servers
kavali.
Kani infrastructure design mostly same.
So modules same use chestham.

---

18. Complete structure

```
terraform/
│
├── environments/
│   │
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
```

```
│   │   └── terraform.tfvars
│   │
│   ├── stage/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   │
│   └── prod/
│       ├── main.tf
│       ├── variables.tf
│       └── terraform.tfvars
│
└── modules/
    │
    ├── vpc/
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    │
    ├── ec2/
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    │
    ├── rds/
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    │
    └── alb/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

Ippudu idi chusi bayapadaku.
Idi just organization.

---

19. modules/vpc
modules/vpc
Meaning:
VPC create cheyyadaniki reusable code.

---

20. modules/ec2
modules/ec2
Meaning:
EC2 servers create cheyyadaniki reusable code.

---

21. modules/rds
modules/rds
Meaning:
Database create cheyyadaniki reusable code.

---

22. modules/alb
modules/alb
Meaning:
Load Balancer create cheyyadaniki reusable code.

---

23. environments/dev
environments/dev
Meaning:
Ee modules ni development environment kosam ela configure cheyyalo.
Example:

```
module "ec2" {
  source = "../../modules/ec2"

  instance_count = 2
  instance_type  = "t3.micro"
}
```

Meaning:

Dev lo EC2 module use cheyyi.
2 servers kavali.
t3.micro size kavali.

---

24. environments/prod
Production lo:

```
module "ec2" {
  source = "../../modules/ec2"

  instance_count = 10
  instance_type  = "t3.large"
}
```

Meaning:
Same EC2 module use cheyyi.
Kani production kosam 10 servers kavali.
Bigger t3.large servers kavali.
Notice:
MODULE CODE
↓
same

DEV VALUES
↓
2 × t3.micro

PROD VALUES
↓
10 × t3.large
Idi modules + environments concept.

---

25. Mari video call / audio call?
Ikkada nee previous doubt important.
Application lo features:
Video Call
Audio Call
Chat
Login
Payment
ivi Terraform modules ani necessarily kaadu.
Terraform ki:
"Video call feature create cheyyi"
ani generally rayamu.
Instead video call feature run avvadaniki infrastructure kavali:
Servers
Networking
Load Balancer
Database
Storage
Monitoring
Terraform aa infrastructure create/manage chestundi.
So:

```
Application
│
├── Video Call
├── Audio Call
├── Chat
├── Login
└── Payment
```

ivi application features.
Terraform side:

```
Infrastructure
│
├── VPC
├── Networking
├── EC2
├── Load Balancer
├── Database
├── Storage
└── Monitoring
```

ivi infrastructure components.

26. Mari ee code maname rayala?
Yes and No.
Real company lo usually 3 situations.
Situation 1
Company already module create chesindi.
Company
↓
modules/vpc already ready
↓
Nuvvu reuse chesthav
Appudu VPC module motham zero nunchi rayalsina avasaram ledu.

Situation 2
Existing module undi, requirement konchem different.
Existing module
↓
Requirement change
↓
Module modify
Appudu existing code ni modify chesthav.

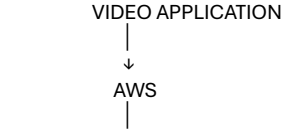
Situation 3
Completely new infrastructure.
New requirement
↓
No existing module
↓
Terraform engineer
↓
New module/code write chestadu

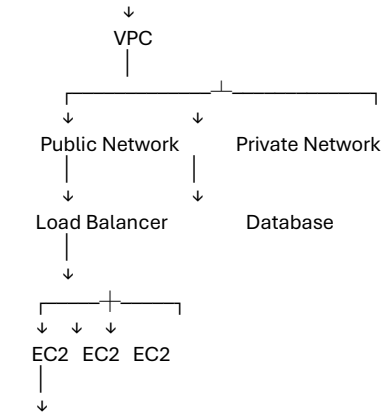
---

27. Real company lo normal workflow
Suppose manager cheppadu:
"Mana video application production AWS lo deploy cheyyali."
DevOps team first check chestundi:
Already VPC module unda?
↓
Yes
↓
Reuse
Already EC2 module unda?
↓
Yes
↓
Reuse
Already RDS module unda?
↓
Yes
↓
Reuse
Then production configuration:
prod/
↓
VPC module
↓
EC2 module
↓
RDS module
↓
ALB module
↓
AWS

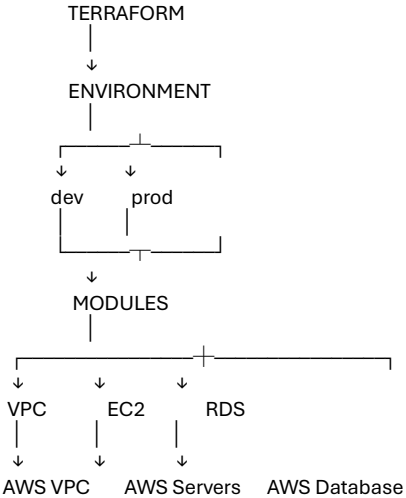
---

28. Final complete picture
Ippudu entire thing ni oka story laga chudu:





Terraform side:



29. One final time — simplest meaning

Terraform
↓
AWS infrastructure ni code tho create/manage cheyyadaniki tool
Resource
↓
Oka actual AWS thing
↓
VPC / EC2 / RDS / S3 / Load Balancer
Module
↓
Reusable Terraform code
variables.tf
↓
Inputs enti?
terraform.tfvars
↓
Inputs values enti?
main.tf
↓
Em infrastructure create/manage cheyyali?
outputs.tf
↓
Create aina infrastructure nunchi em information bayataki kavali?
environments/
↓
Dev / Stage / Prod ni separate ga manage cheyyadam

So nuvvu ippudu Terraform nerchukovadam start chesthe, first VPC → Subnet → Internet Gateway → Route Table → Security Group → EC2 ane AWS basics ardham chesko. Tarvatha vatini Terraform lo ela code chestaro chuste, ee folders/files enduku unnayo automatic ga clear avuthayi.

Terraform folder structure meaning simple ga cheptanu:

```
├── main.tf
├── environment/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   ├── stage/
│   └── prod/
└── modules/
    ├── variables.tf
    └── outputs.tf
```

1. main.tf
Idi root/main Terraform configuration.
Usually modules ni call cheyyadaniki or common configuration kosam use chestaru.

2. environment/
Different environments ni separate ga maintain cheyyadaniki.

- dev/ → Development environment
- stage/ → Testing / staging environment
- prod/ → Production environment

Example:
dev → small/cheap resources
stage → production-like testing
prod → actual production resources

3. environment/dev/main.tf
Dev environment ki kavalsina resources/modules ni define chestundi.
For example:

```
module "app" {
  source = "../../modules"
}
```

Ante modules folder lo unna reusable Terraform code ni dev use chestundi.

4. variables.tf
Terraform lo inputs define cheyyadaniki.
Example:

```
variable "instance_type" {
  type    = string
  default = "t2.micro"
}
```

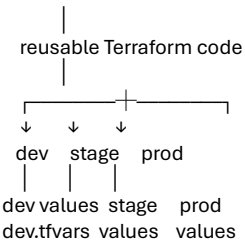
5. terraform.tfvars
Actual variable values ikkada provide chestam.
instance_type = "t2.micro"
So:
variables.tf = variable enti?
terraform.tfvars = variable value enti?

6. modules/
Idi reusable Terraform code kosam.
Suppose AWS EC2 create cheyyadaniki code undi. Dev, Stage, Prod anni same code use cheyyali. Appudu aa code ni module lo petti, different environments nundi call chestam.
Typical module structure:

```
modules/
├── main.tf
├── variables.tf
└── outputs.tf
```

- main.tf → resources create chestundi
- variables.tf → module ki inputs
- outputs.tf → module nundi values bayataki istundi

Simple flow
modules/



Main idea:
modules = common/reusable code
environment/dev = dev-specific configuration
environment/stage = stage-specific configuration
environment/prod = prod-specific configuration

NEXT STEP
1. Terraform ki AWS tho connection ela vastundi?

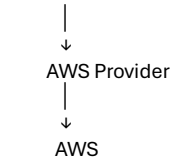
Nuvvu Terraform lo ila code rastav:

```
resource "aws_instance" "app" {
  instance_type = "t3.micro"
}
```

Kani Terraform ki oka doubt:
"aws_instance" ante enti?
AWS tho ela maatladali?
Anduke **Provider**.

---

2. Provider ante enti?
Provider ante simple ga:
Terraform ki AWS tho communicate cheyyadaniki bridge.



Terraform AWS resources create cheyyali ante AWS provider kavali.
Example:

```
provider "aws" {
  region = "ap-south-1"
}
```

Meaning:

```
provider "aws"
    ↓
AWS ni use cheyyali

region
    ↓
Mumbai region lo work cheyyali
```

---

3. First real Terraform project
Munduga modules, dev/stage/prod anni pakkana pettu.
Simple ga **one EC2** create cheddam.
Folder:

```
terraform-demo/
│
├── main.tf
├── variables.tf
├── terraform.tfvars
└── outputs.tf
```

Ippudu prati file actual ga em chestundo chuddam.

---

4. main.tf

```
provider "aws" {
  region = var.region
```

```
}

resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name = var.app_name
  }
}
```

Ikkada rendu important things unnayi.

First:

```
provider "aws" {
  region = var.region
}
```

Meaning:
AWS provider use cheyyi.
Region value var.region nunchi teesuko.

Second:

```
resource "aws_instance" "app"
```

Meaning:
AWS lo EC2 instance resource manage cheyyi.

---

5. variables.tf

```
variable "region" {
  type = string
}

variable "ami_id" {
  type = string
}

variable "instance_type" {
  type = string
}

variable "app_name" {
  type = string
}
```

Idi Terraform ki chepthundi:
Naaku 4 inputs kavali:

region
ami_id
instance_type
app_name

Important:
variables.tf lo values ivvaledu.
Just:
"Ee inputs unnayi."
ani declare chestunnam.

---

6. terraform.tfvars
Ikkada actual values:

```
region       = "ap-south-1"
ami_id       = "YOUR_AMI_ID"
instance_type = "t3.micro"
app_name     = "video-app"
```

Meaning:
region
↓
ap-south-1

ami_id
↓
actual AMI ID

instance_type
↓
t3.micro

app_name
 ↓
video-app
So:
variables.tf
   ↓
"What inputs do I need?"

terraform.tfvars
   ↓
"What are those inputs' values?"

---

7. outputs.tf
output "instance_id" {
 value = aws_instance.app.id
}
Meaning:
EC2 create ayyaka, aa EC2 ID naaku output ga chupinchu.
Example Terraform output:
instance_id = "i-0123456789abcdef"

---

8. Ippudu four files kalipi chudu
main.tf
provider "aws" {
 region = var.region
}

resource "aws_instance" "app" {
 ami   = var.ami_id
 instance_type = var.instance_type

 tags = {
  Name = var.app_name
 }
}
variables.tf
variable "region" {
 type = string
}

variable "ami_id" {
 type = string
}

variable "instance_type" {
 type = string
}

variable "app_name" {
 type = string
}
terraform.tfvars
region   = "ap-south-1"
ami_id   = "YOUR_AMI_ID"
instance_type = "t3.micro"
app_name   = "video-app"
outputs.tf
output "instance_id" {
 value = aws_instance.app.id
}

---

9. Ee 4 files madhya connection enti?
Idi **very important**.
terraform.tfvars
   |
   | values
   ↓
variables.tf
   |
   | variables
   ↓

main.tf
  |
  | infrastructure
  ↓
AWS
  |
  ↓
EC2
  |
  ↓
outputs.tf
  |
  ↓
EC2 ID
Example:
terraform.tfvars

instance_type = "t3.micro"
↓
variables.tf

variable "instance_type"
↓
main.tf

instance_type = var.instance_type
↓
Terraform AWS ki chepthundi:
Create EC2
Type = t3.micro
↓
AWS:
EC2
↓
outputs.tf:
instance_id = i-xxxxxx

---

10. Ippudu actual Terraform commands
Code ready.
Ippudu Terraform ni run cheyyali.
First:
terraform init

---

11. terraform init **ante enti?**
Terraform project first time open chesinappudu:
terraform init
run chestham.
Meaning:
Terraform project ni initialize cheyyi and required provider/plugins prepare cheyyi.
Flow:
main.tf
 ↓
provider "aws"
 ↓
terraform init
 ↓
AWS provider prepare

---

12. Next terraform validate
terraform validate
Meaning:
Mana Terraform configuration syntax/configuration correct ga unda check cheyyi.
Example:
terraform validate
  ↓
Code check
  ↓
Correct?
Correct aithe:
Success! The configuration is valid.

---

13. Next terraform plan
Idi chala important.
terraform plan
Meaning:
AWS lo actual ga changes cheyyakunda, **em changes jaragabotunnayo preview chupinchu.**
Example:
+ aws_instance.app
+ means:
CREATE
Terraform chepthundi:
1 EC2 create cheyyabothunnanu

---

14. Next terraform apply
Plan correct anipisthe:
terraform apply
Meaning:
Preview lo chusina changes ni actual AWS lo execute cheyyi.
Flow:
terraform apply
↓
Terraform
↓
AWS API
↓
EC2 created

---

15. AWS lo final result
AWS
└── ap-south-1
    └── EC2
        ├── Name: video-app
        └── Type: t3.micro

---

16. Work complete ayyaka
Testing kosam create chesunte remove cheyyachu:
terraform destroy
Meaning:
Terraform manage chestunna resources ni AWS nunchi remove cheyyi.
Flow:
terraform destroy
↓
Terraform
↓
AWS
↓
EC2 deleted

---

17. So ippudu Terraform complete basic workflow
WRITE CODE
↓
terraform init
↓
terraform validate
↓
terraform plan
↓
terraform apply
↓
AWS infrastructure
↓
terraform destroy
Idi **Terraform lo basic daily workflow**.

---

18. Next stage enti?
Ippudu manam **EC2 one server create cheyyadam** varaku vacham.
Next actual stage:
ONE EC2
↓

MULTIPLE EC2
↓
count
↓
for_each
↓
variables
↓
conditional expressions
↓
locals
↓
outputs
↓
data sources
Daani tarvatha:
EC2
↓
VPC
↓
Subnet
↓
Security Group
↓
ALB
↓
RDS
↓
S3
Daani tarvatha:
Modules
↓
Dev
↓
Stage
↓
Prod
↓
State
↓
Remote Backend
↓
Git
↓
CI/CD
↓
Real-time project
So **immediate next lesson =** count**,** for_each**, and multiple EC2 servers**. Adi nerchukunte Terraform lo "oka resource ni 1 time kaadu, 2/5/10 times ela create chestaru?" ane real concept start avuthundi.

NEXT STEP

nee exact project structure lo Netflix example petti cheptha. Folder-by-folder **“idi enduku?”** ani chudu.

```
netflix-infrastructure/
│
├── main.tf
│
├── environment/
│   │
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   │
│   ├── stage/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
```

```
│   │
│   └── prod/
│       ├── main.tf
│       ├── variables.tf
│       └── terraform.tfvars
│
└── modules/
      VPC
      EC2
       RDS
       ALB
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```

Ippudu **Netflix AWS infrastructure** ani assume cheddam.

---

1. main.tf — ROOT

```
netflix-infrastructure/
└── main.tf
```

Idi **main/root entry point**.
But nee structure lo environments separate ga unnayi kabatti, practical ga dev, stage, prod folders nunchi Terraform run cheyyachu.
So beginner ga ippudu **root main.tf ni ignore cheyyachu**.

---

2. environment/dev/

```
environment/
└── dev/
    ├── main.tf
    ├── variables.tf
    └── terraform.tfvars
```

Idi **Netflix Developer environment**.
Developers Netflix application ni test chestaru.
dev/main.tf

```
module "netflix" {
 source = "../../modules"

 environment  = "dev"
 instance_type = var.instance_type
}
```

Meaning:
"Modules folder lo unna common Netflix infrastructure code ni teesukoni, DEV environment create cheyyi."

---

dev/variables.tf

```
variable "instance_type" {
 type = string
}
```

Meaning:
Dev server ki **which EC2 size?** ani variable create chestunnam.

---

dev/terraform.tfvars

```
instance_type = "t2.micro"
```

Meaning:
Dev lo t2.micro server use cheyyi.
So:

```
dev/
 │
 ├── main.tf
 │     ↓
 │   modules ni call chestundi
 │
 ├── variables.tf
 │     ↓
 │   "instance_type kavali"
 │
 └── terraform.tfvars
       ↓
    "t2.micro"
```

---

3. environment/stage/

```
environment/
└── stage/
```

```
    ├── main.tf
    ├── variables.tf
    └── terraform.tfvars
```

Idi **Netflix Stage environment**.

stage/main.tf

```
module "netflix" {
  source = "../../modules"

  environment   = "stage"
  instance_type = var.instance_type
}
```

stage/variables.tf

```
variable "instance_type" {
  type = string
}
```

stage/terraform.tfvars

```
instance_type = "t3.medium"
```

So:

```
STAGE
  ↓
modules
  ↓
t3.medium EC2
```

---

4. environment/prod/

```
environment/
└── prod/
    ├── main.tf
    ├── variables.tf
    └── terraform.tfvars
```

Idi **real Netflix production environment**.

prod/main.tf

```
module "netflix" {
  source = "../../modules"

  environment   = "prod"
  instance_type = var.instance_type
}
```

prod/variables.tf

```
variable "instance_type" {
  type = string
}
```

prod/terraform.tfvars

```
instance_type = "t3.large"
```

So:

```
PROD
  ↓
modules
  ↓
t3.large EC2
```

---

5. modules/ — ⭐ Important

Ippudu actual **common/reusable infrastructure code** ikkada untundi.

```
modules/
  ├── main.tf
  ├── variables.tf
  └── outputs.tf
```

modules/main.tf

```
resource "aws_instance" "netflix_server" {
  ami           = "ami-123456"
  instance_type = var.instance_type

  tags = {
    Name        = "netflix-server"
    Environment = var.environment
  }
}
```

Idi actual ga cheptundi:

AWS lo EC2 server create cheyyi.

---

modules/variables.tf

```
variable "environment" {
  type = string
}

variable "instance_type" {
  type = string
}
```

Module cheptundi:
Naaku 2 values kavali:

1. environment
2. instance_type

Dev istundi:
environment = dev
instance_type = t2.micro
Stage istundi:
environment = stage
instance_type = t3.medium
Prod istundi:
environment = prod
instance_type = t3.large

---

6. modules/outputs.tf

```
output "instance_id" {
  value = aws_instance.netflix_server.id
}
```

Idi AWS create chesina EC2 **instance ID ni bayataki return chestundi**.
For example:
i-0123456789abcdef

🔥 **Entire project ni oka flow laga chudu**

DEV ki:

environment/dev/main.tf
↓
source = "../../modules"
↓
modules/main.tf
↓
AWS EC2 CREATE
↓
t2.micro

STAGE ki:

environment/stage/main.tf
↓
modules/main.tf
↓
AWS EC2 CREATE
↓
t3.medium

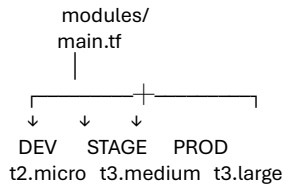
PROD ki:

environment/prod/main.tf
↓
modules/main.tf
↓
AWS EC2 CREATE
↓
t3.large

⭐ **Main point**

environment/dev/main.tf lo EC2 code rayatledu.
environment/stage/main.tf lo kuda EC2 code rayatledu.
environment/prod/main.tf lo kuda EC2 code rayatledu.
Actual EC2 code **one place lo only**:

modules/main.tf
And 3 environments aa code ni reuse chestunnayi:



Modules = common code
environment/dev = dev settings
environment/stage = stage settings
environment/prod = prod settings
terraform.tfvars = actual values
variables.tf = variables declare cheyyadam
outputs.tf = result bayataki ivvadam